

C++ para Gameplay Programmer

Teoría esencial — C++ estándar, sin motor

Cómo usarlo: cada tema tiene la idea en llano (caja azul), la sintaxis, y las trampas típicas (caja naranja).

Cada tema lleva su nivel, que es lo que se espera de un junior:

- **Escribirlo** lo usas cada semana: tienes que escribirlo sin mirar
- **Explicarlo** tienes que saber qué es, por qué existe y cuándo aparece
- **Reconocerlo** si lo ves, sabes qué es; la sintaxis exacta la buscas

Parte 1: el lenguaje. **Parte 2:** librería estándar. **Parte 3:** matemáticas de gameplay. **Parte 4:** preguntas de entrevista con respuesta, ejemplo y el porqué. Repaso semanal: 3-4 preguntas en voz alta.

Las clases de los ejemplos (Weapon, Fighter, Player ...) son inventadas.

Parte 1 — El lenguaje

1. Stack y heap **Explicarlo**

Un programa tiene dos zonas de memoria principales para sus datos:

- **Stack:** cada vez que llamas a una función, se reserva un bloque (un **frame**) con sus variables locales. Al salir de la función, ese bloque se libera entero de golpe. Por eso las variables locales se crean y se destruyen **solas**, y por eso es tan rápido: reservar y liberar es solo mover un puntero interno. El tamaño de cada variable se conoce al compilar, y el stack es pequeño (en Windows, ~1 MB por defecto).
- **Heap:** una zona grande de la que pides memoria **en runtime**, del tamaño que quieras, con `new`. Esa memoria no está atada a ninguna función: sigue reservada hasta que alguien hace `delete`. Es más lento, porque el sistema tiene que buscar un hueco libre y llevar la cuenta.

	Stack	Heap
Qué va ahí	Variables locales de las funciones	Lo que creas con <code>new</code>
Cuánto vive	Hasta el <code>}</code> de su scope	Hasta que alguien hace <code>delete</code>
Quién lo libera	Se libera solo	Tú (con <code>delete</code> , un destructor o un smart pointer)
Tamaño	Fijo y conocido al compilar; zona pequeña	Decidido en runtime; zona grande
Velocidad	Muy rápido	Más lento

```
void Foo()
{
    int X = 5;           // stack
    int* P = new int(10); // P (el puntero) en el stack; el 10, en el heap
}                       // mueren X y P... pero el 10 sigue en el heap
```

Antes del }:
Después:

P (stack) —> [10] (heap)
[10] (heap) <- nadie sabe dónde está: LEAK

Memory leak = memoria reservada a la que ya nadie puede llegar. Pasa cuando muere el último puntero que sabía dónde estaba.

Regla: usa el stack siempre que puedas. El heap solo si el objeto tiene que vivir más que su función, si la cantidad se decide en runtime o si hay polimorfismo.

Trampa: en C++ moderno casi nunca escribes `new` ni `delete` a mano: usas smart pointers (temas 9 y 10).

2. Punteros Escribirlo

Un puntero es una variable que guarda una **dirección de memoria**: el "número de estantería" donde está algo. `nullptr` = no apunta a nada.

```
int Health = 100;
int* Ptr = &Health;    // &Health = "la dirección de Health"
*Ptr = 50;              // *Ptr    = "lo que hay en esa dirección" -> Health vale 50

Enemy* E = FindEnemy();
if (E != nullptr)      // comprobar SIEMPRE antes de usar
{
    E->TakeDamage(10);  // -> es un atajo de (*E).TakeDamage(10)
}
```

Trampa: puntero colgante: apunta a algo que ya se destruyó. Usarlo es comportamiento indefinido. Caso típico: guardas un puntero a un elemento de un `vector` y luego haces `push_back`; el vector puede mudar todos sus elementos a otra zona de memoria.

3. Referencias Escribirlo

Una referencia es **otro nombre** para una variable que ya existe. No es una copia: es la misma variable. No puede ser null y no se puede cambiar a qué se refiere.

```
void Heal(int& Health)    // recibe el ORIGINAL, no una copia
{
    Health += 10;
}

int PlayerHealth = 90;
Heal(PlayerHealth);      // PlayerHealth = 100

int& Alias = PlayerHealth; // obligatorio darle valor al declararla
```

Usa...	Cuando...
Referencia <code>T&</code>	el objeto seguro que existe
Puntero <code>T*</code>	puede no existir (<code>nullptr</code>) o necesitas cambiar a qué apunta

Trampa: nunca devuelvas una referencia a una variable local: la variable muere al salir de la función.

```
int& Bad()
{
    int X = 5;
    return X;    // X muere aquí -> la referencia apunta a basura
}
```

4. Paso de parámetros Escribirlo

Firma	Qué recibe la función	Úsalo para
<code>void F(Item I)</code>	una copia	tipos pequeños: <code>int</code> , <code>float</code> , <code>bool</code> , punteros
<code>void F(Item& I)</code>	el original , puede cambiarlo	cuando la función tiene que modificarlo
<code>void F(const Item& I)</code>	el original , solo lectura, sin copia	lo normal para clases y <code>std::string</code>
<code>void F(Item* I)</code>	un puntero, puede ser <code>nullptr</code>	parámetros opcionales
<code>void F(std::unique_ptr<Item> I)</code>	la propiedad	cuando la función se queda con el objeto

5. const Escribirlo

`const` es una promesa que vigila el compilador: "esto no va a cambiar". Si la rompes, no compila. Es una red de seguridad gratis.

```
const int MaxHealth = 100;    // variable que no cambia

void Print(const Item& I);    // parámetro de solo lectura

float GetHealth() const;    // método que promete no modificar el objeto
```

Con punteros hay dos `const` distintos. Truco: se lee de derecha a izquierda.

```
const int* A;    // "A es un puntero a un int const": no puedo cambiar el valor a través de A
int* const B = &X;    // "B es un puntero const a int": B no puede apuntar a otro sitio
```

En un getter también hay dos `const` distintos:

```
const std::string& GetName() const;
// ^ lo que devuelvo no se puede modificar
//           ^ esta función no modifica el objeto
```

Trampa: sobre un objeto `const` (o recibido por `const&`) **solo** puedes llamar a métodos `const`. Si olvidas el `const` de un getter, no podrás usarlo.

Trampa: `const` protege el objeto, no lo que apuntan sus punteros. Un método `const` puede devolver un `Fighter*` no `const`.

6. Clases Escribirlo

```
class Weapon
{
public:
    explicit Weapon(const std::string& name, float damage) // constructor
        : Name(name) // lista de inicialización
        , Damage(damage)
    {
    }

    virtual ~Weapon() { } // destructor: se ejecuta solo al
morir

    const std::string& GetName() const { return Name; }

private:
    std::string Name;
    float Damage = 0.0f; // valor por defecto
};

Weapon W("Katana", 25.0f); // objeto en el stack
```

Acceso	Quién puede usarlo
public	cualquiera
protected	la clase y sus hijas
private	solo la propia clase (las hijas no)

- **Lista de inicialización** (`: Name(name)`): construye los miembros directamente. Es obligatoria para miembros `const`, referencias y para llamar al constructor del padre.
- **Dentro de la clase** usa los miembros directamente (`Name`), no hace falta el getter.
- `struct` es igual que `class`, pero `public` por defecto. Convención: `struct` para datos simples.

Trampa: los miembros se inicializan en el orden en que están **declarados en la clase**, no en el de la lista.

Trampa: un `int`, `float`, `bool` o puntero miembro sin inicializar tiene **basura**. Dale siempre valor por defecto.

explicit

Un constructor de **un solo parámetro** hace dos trabajos: construir, y servir de **conversión automática**. El compilador lo usa por su cuenta, sin avisar:

```
Weapon(float damage);           // SIN explicit
void Equip(const Weapon& w);

Equip(50.0f);                   // compila: el compilador se inventa un Weapon a partir de 50.0f
```

```
explicit Weapon(float damage); // CON explicit

Equip(50.0f);                   // ERROR
Equip(Weapon(50.0f));           // OK: lo pides tú, a la vista
```

Regla: pon `explicit` en todo constructor de un parámetro, salvo que quieras la conversión a propósito (como `std::string`, que permite `std::string S = "hola";`).

7. Herencia y virtual Escribirlo

Un hijo **contiene** al padre dentro. Primero se construye la parte del padre y encima la del hijo. Al destruir, al revés: primero el hijo y luego el padre.

```
class Weapon
{
public:
    Weapon(const std::string& name, float damage);
    virtual ~Weapon() = default;           // destructor virtual: OBLIGATORIO si hay herencia
    virtual void Use() const;               // los hijos pueden reemplazarla
    virtual void Special() = 0;             // virtual PURA: los hijos DEBEN implementarla
};                                          // -> Weapon es abstracta: no se puede crear

class Sword : public Weapon
{
public:
    Sword(const std::string& name, float damage)
        : Weapon(name, damage)             // el hijo construye al padre en la lista
    {
    }

    void Use() const override;              // override: "reemplazo una virtual del padre"
    void Special() override;

};

std::vector<std::unique_ptr<Weapon>> Weapons;
Weapons.push_back(std::make_unique<Sword>("Katana", 25.0f));

for (auto& W : Weapons)
{
    W->Use();                              // llama a Sword::Use (polimorfismo)
}
```

	Qué versión se llama	Cuándo se decide
Con <code>virtual</code>	la del objeto real (Sword)	en runtime
Sin <code>virtual</code>	la del tipo del puntero (Weapon)	al compilar

- Llamar a la versión del padre desde el hijo: `Weapon::Use()`;
- El hijo **no** puede tocar los miembros `private` del padre: usa sus getters o el constructor del padre.
- Si no hay relación "es un", mejor **composición** (tener un miembro) que herencia.

Trampa: destructor `sin virtual` y borrar un hijo a través de un puntero al padre: solo se ejecuta el destructor del padre (comportamiento indefinido). `unique_ptr<Weapon>` que guarda un `Sword` hace exactamente eso.

Trampa: `sin override`, si te equivocas en la firma creas una función nueva sin enterarte. Con `override`, el compilador te avisa.

Trampa: slicing: copiar un hijo en una variable del padre **por valor** corta la parte del hijo. Con herencia, trabaja con punteros o referencias.

8. RAII [Explicarlo](#)

Como la luz de un pasillo con sensor: entras y se enciende, sales y se apaga sola. El recurso se pide en el **constructor** y se libera en el **destructor**, así que se libera solo al morir el objeto, aunque haya `return` tempranos.

La idea en una frase: **el recurso vive lo mismo que el objeto que lo guarda**. (Si vienes de Java: es como `try-with-resources`, pero automático.)

```
void LoadGame()
{
    SaveData* Save = new SaveData("slot1.sav");
    if (Save->IsCorrupted())
    {
        return;          // LEAK: nunca llegamos al delete
    }
    delete Save;
}
```

```
class ScopedSave
{
public:
    explicit ScopedSave(const char* path) : Data(new SaveData(path)) {} // adquiere
    ~ScopedSave() { delete Data; } // libera
    SaveData* Get() const { return Data; }
private:
    SaveData* Data;
};

void LoadGame()
{
    ScopedSave Save("slot1.sav"); // vive en el stack
    if (Save.Get()->IsCorrupted())
    {
        return;          // ~ScopedSave() se ejecuta solo -> sin leak
    }
}
```

La librería estándar entera funciona así: `std::vector`, `std::string`, `std::unique_ptr`... Nunca haces `delete` de un `vector`.

Trampa: si copias un `ScopedSave`, las dos copias apuntan al mismo `SaveData` y las dos hacen `delete`. Solución: prohibir la copia (tema 11) o usar `unique_ptr`.

9. `unique_ptr` [Escribirlo](#)

Un puntero con **un solo dueño**. El dueño es el responsable de borrar. Cuando el `unique_ptr` muere, borra el objeto. Es RAII ya hecho.

```
#include <memory>

std::unique_ptr<Weapon> A = std::make_unique<Sword>("Katana", 25.0f); // nunca new
A->Use(); // se usa como un puntero normal

std::unique_ptr<Weapon> B = A; // ERROR: no se puede copiar
std::unique_ptr<Weapon> C = std::move(A); // OK: se TRANSFIERE. C es el dueño, A queda vacío

Weapon* Raw = C.get(); // puntero normal para usarlo, sin soltar la
propiedad
PrintWeapon(*C); // *C = el objeto, para funciones que piden Weapon&
if (C) { } // ¿tiene algo dentro?
C.reset(); // borra el objeto ya
```

Tipo	Significa
<code>std::unique_ptr<T></code>	"soy el dueño"
<code>T*</code> o <code>T&</code>	"solo lo uso, no lo borro"

- Una función que se va a **quedar** con él lo recibe **por valor**: `void Add(std::unique_ptr<T> item)`.
- Una función que solo lo **usa** recibe `const T&` o `T*`, no el `unique_ptr`.
- `std::move` solo hace falta si lo que mueves **tiene nombre**. `push_back(std::make_unique<T>(...))` no lo necesita: es un temporal.
- `std::vector<std::unique_ptr<T>>`: si el vector crece, se mudan los `unique_ptr`, no los objetos. Los `T*` que repartiste siguen siendo válidos.

Trampa: usar un `unique_ptr` después de moverlo: está vacío (`nullptr`).

10. `shared_ptr` y `weak_ptr` [Explicarlo](#)

shared_ptr: varios dueños con un contador. El objeto muere cuando el **último** lo suelta. **weak_ptr**: como tener el **número de teléfono** de alguien. Puedes llamarle, pero no eres responsable de él, y quizá ya no conteste.

```
std::shared_ptr<Player> P1 = std::make_shared<Player>("Wolf"); // contador = 1
std::shared_ptr<Player> P2 = P1; // se PUEDE copiar -> contador =
2
P1.use_count(); // 2
P1.reset(); // contador = 1, Wolf sigue vivo
```

Un `weak_ptr` no se usa directamente (no tiene `->`). Para usarlo, `lock()` : te devuelve un `shared_ptr` lleno si el objeto sigue existiendo, o vacío si ya murió.

```
class Enemy
{
public:
    void SetTarget(std::weak_ptr<Player> target) { Target = target; }

    void Attack() const
    {
        if (std::shared_ptr<Player> Locked = Target.lock())    // "llamo por teléfono"
        {
            std::cout << "Ataca a " << Locked->GetName() << "\n";    // uso Locked, no Target
        }
        else
        {
            std::cout << "Sin objetivo\n";                            // el jugador ya no existe
        }
    }

private:
    std::weak_ptr<Player> Target;    // lo observa, NO lo mantiene vivo
};

Enemy E;
E.SetTarget(P2);    // P2 es shared_ptr: se convierte solo a weak_ptr, el contador NO sube
```

Fuera del if: P2 ———→ [Player] contador = 1
Target ..] (weak: no cuenta)

Dentro del if: P2 ———→
Locked ———→ [Player] contador = 2 (un solo Player, dos dueños)
Target ..]

Al salir del if, Locked muere -> contador = 1

- **"Vivo"** aquí significa "el objeto existe en memoria", no "le queda vida". Son cosas distintas.
- `Locked` es un dueño temporal: mientras dura el `if`, el jugador no puede destruirse a mitad de uso.

Situación	Qué usar
Vive lo que dura el scope y nadie lo apunta	stack
Heap con un solo dueño	<code>unique_ptr</code> (el de por defecto)
Varios dueños de verdad, o alguien lo observa con <code>weak_ptr</code>	<code>shared_ptr</code>
Observar sin mantener vivo (el objetivo de un enemigo)	<code>weak_ptr</code>

Trampa: si el enemigo guardara un `shared_ptr` al jugador, el jugador **no moriría** al hacer `reset()` : observar lo mantendría vivo artificialmente.

Trampa: ciclo: A tiene un `shared_ptr` a B y B a A. El contador nunca llega a 0: ninguno se borra. Uno de los dos debe ser `weak_ptr`.

11. Copia y move Explicarlo

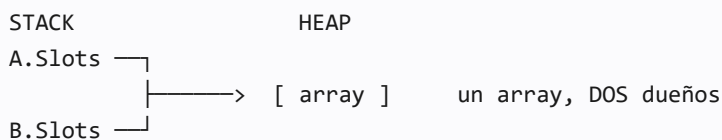
Al crear o asignar un objeto a partir de otro hay dos opciones. **Copiar**: duplicar los datos, y los dos quedan completos. **Mover**: llevarte los datos del otro y dejarlo vacío. Mover es mucho más barato.

El problema: copiar un puntero crudo

La copia que genera el compilador copia cada miembro tal cual. Si un miembro es un puntero, copia **la dirección**, no lo que hay en el heap:

```
class Inventory
{
    int* Slots;    // apunta a un array en el heap
};

Inventory A;
Inventory B = A;  // B.Slots = A.Slots
```



Al final mueren A y B -> los dos hacen delete[] del mismo array -> DOUBLE DELETE

Con `std::vector` no pasa: el vector sabe copiarse bien. Cada copia reserva su propio array.

Con vector: A -> [1][2][3]
 B -> [1][2][3] dos arrays, un dueño cada uno

Las 5 funciones especiales

Controlan qué pasa al copiar, mover y destruir. Si no las escribes, el compilador las genera solas.

Función	Cuándo se llama	Qué debe hacer
Destructor	A muere	borrar lo suyo del heap
Constructor de copia	Inventory B = A; (B es nuevo)	crear su propio array y copiar los valores
Asignación por copia	B = A; (B ya existía)	crear un array nuevo, copiar, borrar el viejo
Constructor de move	Inventory B = std::move(A);	quedarse el array de A y dejar A vacío
Asignación por move	B = std::move(A); (B ya existía)	borrar el suyo, quedarse el de A y dejar A vacío

Trampa: `Inventory B = A;` **no** es una asignación, aunque lleve `=`. B no existía: se llama al **constructor** de copia. Solo es asignación si el objeto ya existía antes.

El código (nivel: reconocerlo)

```
class Inventory
{
public:
    explicit Inventory(size_t size) : Size(size), Slots(new int[size]{{}}) {}

    // 1. Destructor
    ~Inventory() { delete[] Slots; } // new[] se libera con delete[]

    // 2. Constructor de copia
    Inventory(const Inventory& other)
        : Size(other.Size)
        , Slots(new int[other.Size]) // MI PROPIO array
    {
        std::copy(other.Slots, other.Slots + other.Size, Slots); // desde, hasta, destino
    }

    // 3. Asignación por copia
    Inventory& operator=(const Inventory& other)
    {
        if (this != &other) // protege de A = A
        {
            int* NewSlots = new int[other.Size];
            std::copy(other.Slots, other.Slots + other.Size, NewSlots);
            delete[] Slots; // borro mi array viejo
            Slots = NewSlots;
            Size = other.Size;
        }
        return *this;
    }

    // 4. Constructor de move
    Inventory(Inventory&& other) noexcept
        : Size(other.Size)
        , Slots(other.Slots) // me quedo SU puntero: no copio nada
    {
        other.Size = 0;
        other.Slots = nullptr; // el otro queda vacío
    }

    // 5. Asignación por move
    Inventory& operator=(Inventory&& other) noexcept
    {
        if (this != &other)
        {
            delete[] Slots; // borro lo mío (si no: leak)
            Slots = other.Slots; // me quedo lo suyo
            Size = other.Size;
            other.Slots = nullptr; // el otro queda vacío
            other.Size = 0;
        }
        return *this;
    }
}
```

```
private:
    size_t Size = 0;
    int* Slots = nullptr;
};
```

Las piezas de sintaxis

- `**operator=` es un método normal con un nombre especial. `B = A;` se traduce a `B.operator=(A);`: B llama, A es `other`.
- `**this` es un puntero al objeto que llama al método. En `B = A`, `this` apunta a B. `**this` es B en sí.
- `**return *this;` devuelve el propio B (por referencia, por el `&` de `Inventory&`). Permite encadenar `C = B = A;`, igual que con los `int`.
- `**Inventory&&` = "algo que se puede vaciar". `std::move(A)` **no mueve nada**: solo convierte A a ese tipo. El trabajo lo hacen las funciones 4 y 5.
- `**noexcept` promete que no lanza excepciones. `std::vector` solo usa tu `move` al crecer si es `noexcept`; si no, copia.
- `**¿Por qué other.Slots = nullptr?*` Porque cuando `other` muera, su destructor hará `delete[]`. Si siguiera apuntando al array, borraría el array que ahora es tuyo.

Las reglas

Regla	Cuándo
Regla de 0 (lo ideal)	tus miembros ya se gestionan solos (<code>vector</code> , <code>string</code> , <code>unique_ptr</code>). No escribes ninguna de las 5.
Regla de 3 / 5	gestionas un recurso a mano. Si escribes una, casi seguro necesitas todas.

```
class Inventory // regla de 0: la misma clase, sin nada de lo anterior
{
public:
    explicit Inventory(size_t size) : Slots(size) {}
private:
    std::vector<int> Slots;
};
```

Prohibir la copia y permitir el move (así está hecho `unique_ptr`):

```
class UniqueSave
{
public:
    UniqueSave(const UniqueSave&) = delete; // copia prohibida
    UniqueSave& operator=(const UniqueSave&) = delete;

    UniqueSave(UniqueSave&&) noexcept = default; // move: vale la versión del compilador
    UniqueSave& operator=(UniqueSave&&) noexcept = default;
};
```

12. Lambdas Escribirlo

Una función **sin nombre** que escribes justo donde la necesitas. Puede **capturar** variables de alrededor para usarlas dentro.

```
// [captura](parámetros) { cuerpo }
auto IsDead = [](const Fighter& f) { return !f.IsAlive(); };
IsDead(Wolf);                                // se llama como una función

float MinHealth = 20.0f;
auto IsLow = [MinHealth](const Fighter& f)    // captura MinHealth por copia
{
    return f.GetHealth() < MinHealth;
};

int Count = 0;
auto Increment = [&Count]() { ++Count; };    // captura Count por referencia: modifica el
original
```

Captura	Qué captura
<code>[]</code>	nada
<code>[x]</code>	x por copia
<code>[&x]</code>	x por referencia
<code>[=]</code> / <code>[&]</code>	todo lo que uses, por copia / por referencia
<code>[this]</code>	el objeto actual (para usar sus miembros)

Para guardar una lambda como miembro, o pasarla a una función, se usa `std::function` (`#include <functional>`). Entre `< >` va la firma: `retorno(parámetros)`.

```
class Fighter
{
public:
    std::function<void(float)> OnDamaged;    // un "callback": alguien lo rellena desde fuera

    void TakeDamage(float amount)
    {
        Health -= amount;
        if (OnDamaged)                    // ¿alguien lo ha rellenado?
        {
            OnDamaged(amount);
        }
    }

private:
    float Health = 100.0f;
};

Wolf.OnDamaged = [](float amount) { std::cout << "Ay: " << amount << "\n"; };
```

Trampa: capturar por referencia (o `this`) y ejecutar la lambda **más tarde**, cuando esa variable u objeto ya ha muerto: dangling. Típico en callbacks y timers.

13. Sobrecarga de operadores [Reconocerlo](#)

Puedes definir qué hacen `+`, `==`, `<<` ... con tus clases. `A + B` se traduce a `A.operator+(B)`. Muy usado en matemática de juegos.

```
struct Vec2
{
    float X = 0.0f;
    float Y = 0.0f;

    Vec2 operator+(const Vec2& other) const { return { X + other.X, Y + other.Y }; }
    Vec2 operator-(const Vec2& other) const { return { X - other.X, Y - other.Y }; }
    Vec2 operator*(float s) const           { return { X * s, Y * s }; }
};

Vec2 Pos = Vec2{ 1.0f, 2.0f } + Vec2{ 3.0f, 4.0f };    // { 4, 6 }
```

Trampa: comparar `float` con `==` falla por redondeo. Se compara con un margen: `std::abs(A - B) < 0.0001f`.

14. Templates [Explicarlo](#)

Código que sirve para cualquier tipo. Tú escribes una versión con `T` y el compilador genera una por cada tipo que uses.

```
template <typename T>
T Clamp(T value, T min, T max)
{
    if (value < min) return min;
    if (value > max) return max;
    return value;
}

Clamp(150, 0, 100);           // T = int    -> 100
Clamp(1.5f, 0.0f, 1.0f);     // T = float  -> 1.0f
Clamp(1.5f, 0, 1);           // ERROR: T no puede ser float e int a la vez

template <typename T>
class Pool
{
public:
    void Add(const T& item) { Items.push_back(item); }
private:
    std::vector<T> Items;
};

Pool<int> Numbers;             // el tipo va entre < >
```

Ya los usas sin darte cuenta: `std::vector<int>`, `std::unique_ptr<Weapon>` y `std::max` son templates.

Trampa: los templates se escriben ****enteros** en el `.h`. Si los defines en el `.cpp`: error de linker.

Trampa: `std::max(Health - amount, 0)` no compila si `Health` es `float`: el template exige el **mismo tipo** en los dos. Escribe `0.0f`.

15. enum class y struct Escribirlo

```
enum class EState { Idle, Attacking, Blocking, Dead };

EState State = EState::Idle;

switch (State)
{
    case EState::Idle:      break;
    case EState::Attacking: break;
    default:                break;
}

int AsInt = static_cast<int>(State); // no se convierte a int sola

struct HitInfo
{
    float Damage = 0.0f;
    bool bIsCritical = false;
};

HitInfo Hit{ 25.0f, true };
```

Regla: usa siempre `enum class`, no `enum`: no se convierte sola a `int` y sus nombres no se mezclan con el resto del código.

16. Casts Escribirlo

```
float Speed = 3.7f;
int Rounded = static_cast<int>(Speed); // 3. Conversiones normales entre tipos

Weapon* W = GetWeapon();
Sword* S = dynamic_cast<Sword*>(W); // nullptr si no es una Sword
if (S != nullptr) { } // (la clase debe tener algo virtual)
```

- Evita el cast estilo C `(int)Speed`: hace cualquier cosa sin avisar.
- `const_cast` y `reinterpret_cast`: casi nunca en gameplay.

17. static [Explicarlo](#)

```
class Fighter
{
public:
    static int Count;           // UNO solo, compartido por todos los Fighter
    static void PrintCount();   // se llama sin objeto: Fighter::PrintCount();
};

int Fighter::Count = 0;        // se define en el .cpp

int NextId()
{
    static int Id = 0;         // se inicializa una vez y conserva su valor entre llamadas
    return ++Id;
}
```

Trampa: static es estado global: difícil de testear y de depurar. Úsalo poco.

18. .h, .cpp y compilación [Explicarlo](#)

Pieza	Qué es
.h	declaraciones: qué existe
.cpp	definiciones: cómo funciona
#pragma once	evita que un .h se incluya dos veces
Compilador	compila cada .cpp por separado
Linker	une todo al final

- **Forward declaration** (class Player; en el .h): basta si solo usas Player*, Player& o un smart pointer. El #include va en el .cpp. Compila más rápido y evita includes circulares.
- Necesitas el #include completo si heredas de la clase, la tienes como miembro por valor o llamas a sus métodos.
- **Error de compilador:** sintaxis o tipos dentro de un archivo.
- **Error de linker** (unresolved external symbol): algo está declarado pero no aparece su definición. Causas: falta el cuerpo de la función, el .cpp no está en el proyecto o la firma no coincide entre .h y .cpp.

Trampa: una clase con un miembro std::unique_ptr<T> y T solo forward-declarado: su destructor necesita ver T completo. Si da error, declara el destructor en el .h y defínelo en el .cpp con = default.

19. Comportamiento indefinido (UB) [Explicarlo](#)

El estándar no dice qué pasa. Puede funcionar, crashear o corromper memoria en silencio y fallar mucho después, en otro sitio. Por eso es tan peligroso.

- Usar un puntero colgante o un nullptr.

- Leer fuera de un array. `vector[i]` no comprueba nada; `vector.at(i)` sí.
- Leer una variable sin inicializar.
- Hacer `delete` dos veces, o de algo que no vino de `new`.
- Borrar un hijo a través de un puntero al padre sin destructor `virtual`.
- Desbordar un `int` con signo.
- Añadir o borrar elementos de un `vector` mientras lo recorres con un range-for.

Parte 2 — Librería estándar

20. Complejidad: la notación Big-O [Explicarlo](#)

Big-O dice **cómo crece el coste de una operación cuando crece el número de elementos (n)**. No mide segundos: mide cuántos pasos más hacen falta con 10, 1.000 o un millón de elementos.

Notación	Nombre	Qué significa	Ejemplo
O(1)	constante	el coste no depende de n	<code>V[5]</code> en un vector: va directo a la posición
O(log n)	logarítmica	cada paso descarta la mitad de lo que queda	buscar en un <code>std::map</code> (árbol)
O(n)	lineal	el coste crece igual que n	<code>std::find_if</code> en un vector: mira uno por uno
O(n²)	cuadrática	por cada elemento, recorres todos otra vez	dos <code>for</code> anidados sobre la misma lista

n (elementos)	O(1)	O(log n)	O(n)	O(n²)
1.000	1	~10	1.000	1.000.000
1.000.000	1	~20	1.000.000	1.000.000.000.000

Ejemplo de gameplay: 500 enemigos y quieres saber qué pares están cerca entre sí. Comparar cada uno con todos es $O(n^2)$: unas 250.000 comparaciones **por frame**. Por eso los motores dividen el mundo en zonas y solo comparan a los que están en la misma.

Trampa: Big-O dice cómo **crece** el coste, no qué es más rápido con pocos elementos. Con 50 elementos, buscar $O(n)$ en un `vector` suele ganar a otras estructuras, porque su memoria seguida se lee muy rápido.

"**De media**" (como en `unordered_map`) significa que casi siempre es $O(1)$, aunque en casos raros puede ser más lento.

21. Contenedores [Escribirlo](#)

Contenedor	Qué es	Buscar
<code>std::vector<T></code>	array dinámico, memoria seguida. El de por defecto	$O(n)$
<code>std::array<T, N></code>	tamaño fijo, conocido al compilar	$O(n)$
<code>std::unordered_map<K, V></code>	clave → valor, tabla hash, sin orden	$O(1)$ de media
<code>std::map<K, V></code>	clave → valor, ordenado por clave	$O(\log n)$
<code>std::unordered_set<T></code> / <code>std::set<T></code>	solo claves, sin repetidos	$O(1)$ / $O(\log n)$

```

std::vector<int> V = { 1, 2, 3 };
V.push_back(4);
V.size();           // 4
V[0];               // 1 (sin comprobar el rango)
V.reserve(100);     // reserva de golpe: evita mudanzas

std::unordered_map<std::string, int> Ammo;
Ammo["Arrows"] = 20; // inserta o sobrescribe

auto It = Ammo.find("Bombs");
if (It != Ammo.end()) // ¿encontrado?
{
    It->first;         // clave
    It->second;        // valor
}

```

Trampa: `Ammo["clave"]` **inserta** la clave si no existe. Para solo consultar, usa `find`.

Trampa: `push_back` puede mudar el vector a otra zona de memoria: invalida punteros, referencias e iteradores a sus elementos.

¿Por qué `vector` casi siempre? La memoria seguida se lee muy rápido (caché de la CPU). Una lista enlazada suele ser más lenta en la práctica aunque en teoría inserte más rápido.

22. Range-for e iteradores Escribirlo

```

for (const Fighter& f : Fighters) { } // leer sin copiar
for (Fighter& f : Fighters) { }       // modificar
for (auto& f : Fighters) { }          // igual, con auto

for (auto It = V.begin(); It != V.end(); ++It)
{
    *It;                             // el elemento
}

```

`begin()` = primer elemento. `end()` = una posición **después** del último: nunca se desreferencia.

Trampa: `for (Fighter f : Fighters)` sin `&` **copia** cada elemento.

23. Algoritmos Escribirlo

```
#include <algorithm>

auto It = std::find_if(V.begin(), V.end(),
    [](const Fighter& f) { return f.IsAlive(); }); // el primero que cumpla
if (It != V.end()) { }

std::sort(V.begin(), V.end(),
    [](const Fighter& a, const Fighter& b) { return a.GetHealth() < b.GetHealth(); });

auto Alive = std::count_if(V.begin(), V.end(),
    [](const Fighter& f) { return f.IsAlive(); });

std::erase_if(V, [](const Fighter& f) { return !f.IsAlive(); }); // C++20: borra los que
cumplan
```

Si el vector guarda `unique_ptr`, la lambda recibe lo que guarda el vector: `const std::unique_ptr<Fighter>& f`, y se usa con `f->`.

Trampa: `std::remove_if` **no borra**: mueve los elementos al final. Hace falta `erase` después. En C++20, `std::erase_if` lo hace todo.

24. auto Escribirlo

```
auto X = 5; // int
auto It = Ammo.find(Key); // tipos largos: auto ayuda a leer
const auto& First = V[0]; // referencia const, sin copia
```

Trampa: `auto` quita el `const` y el `&`. `auto X = V[0]`; hace una **copia**. Si no la quieres, escribe `const auto&` o `auto&`.

Parte 3 — Matemáticas de gameplay

Todo en C++ puro, con un `struct Vec3` propio (y el `Vec2` del tema 13 para los ejemplos en 2D). Los motores traen su versión, con más funciones, pero hacen exactamente esto.

25. Vectores Escribirlo

Un vector son tres números (X, Y, Z). Se usa para dos cosas distintas: una **posición** (dónde está algo) o una **dirección con tamaño** (hacia dónde y cuánto: una velocidad, un desplazamiento).

```
#include <cmath>

struct Vec3
{
    float X = 0.0f;
    float Y = 0.0f;
    float Z = 0.0f;

    Vec3 operator+(const Vec3& o) const { return { X + o.X, Y + o.Y, Z + o.Z }; }
    Vec3 operator-(const Vec3& o) const { return { X - o.X, Y - o.Y, Z - o.Z }; }
    Vec3 operator*(float s) const      { return { X * s, Y * s, Z * s }; }

    float LengthSquared() const { return X * X + Y * Y + Z * Z; }
    float Length() const       { return std::sqrt(LengthSquared()); }

    Vec3 Normalized() const
    {
        const float L = Length();
        if (L < 0.0001f)
        {
            return {}; // vector cero: no tiene dirección
        }
        return { X / L, Y / L, Z / L };
    }
};
```

Operación	Qué significa	Ejemplo
<code>A + B</code>	mover A según B	<code>Pos + Velocity</code> : la posición siguiente
<code>B - A</code>	el vector de A hacia B	<code>Player - Enemy</code> : del enemigo hacia el jugador
<code>V * 2</code>	mismo sentido, el doble de largo	<code>Dir * Speed</code> : velocidad a partir de una dirección
<code>Length()</code>	cuánto mide (con Pitágoras)	<code>(Player - Enemy).Length()</code> : distancia entre los dos
<code>Normalized()</code>	misma dirección, longitud 1	solo quiero hacia dónde, no cuánto

El ejemplo más típico, un enemigo que persigue al jugador:

```
Vec3 ToPlayer = Player - Enemy;           // de Enemy hacia Player
Vec3 Dir = ToPlayer.Normalized();          // solo la dirección (longitud 1)
Enemy = Enemy + Dir * Speed * DeltaTime; // avanza Speed unidades por segundo
```

Trampa: normalizar un vector de longitud 0 divide entre 0. Compruébalo antes (como hace `Normalized()` arriba).

Trampa: para **comparar** distancias usa `LengthSquared()`: si $A^2 < B^2$, entonces $A < B$, y te ahorras la raíz cuadrada. `DistSq < Range * Range`.

Qué eje es "arriba" (Y o Z) depende del motor. Las fórmulas son las mismas.

26. DeltaTime Escribirlo

`DeltaTime` son los **segundos que han pasado desde el frame anterior**. Si mueves algo una cantidad fija por frame, a más FPS se moverá más rápido. Multiplicando por `DeltaTime`, las velocidades pasan a ser **por segundo**, y el juego va igual a 30 que a 120 FPS.

```
// MAL: 5 unidades por FRAME -> a 60 FPS, 300 por segundo; a 120 FPS, 600
Pos.X += 5.0f;

// BIEN: 300 unidades por SEGUNDO, sea cual sea el framerate
Pos.X += 300.0f * DeltaTime; // a 60 FPS, DeltaTime = 0.0166 -> 5 por frame
```

Regla: todo lo que cambia con el tiempo (movimiento, rotación, regeneración, cooldowns) se multiplica o se descuenta con `DeltaTime`.

27. Dot product (producto escalar) Escribirlo

Toma dos vectores y devuelve **un número** que dice **cuánto apuntan hacia el mismo lado**. Con los dos vectores normalizados, ese número va de **1** (misma dirección) a **-1** (opuestos), y **0** si son perpendiculares.

```
float Dot(const Vec3& a, const Vec3& b)
{
    return a.X * b.X + a.Y * b.Y + a.Z * b.Z;
}
```

dot = 1	dot = 0	dot = -1
→ →	→	→ ←
mismo lado	↑ perpendicular	opuestos

¿El objetivo está delante o detrás de mí?

```
Vec3 ToTarget = (Target - Me).Normalized();
float D = Dot(MyForward, ToTarget); // MyForward = hacia dónde miro (normalizado)

if (D > 0.0f) { /* delante */ }
if (D < 0.0f) { /* detrás */ }
```

Cono de visión: el dot de dos vectores normalizados es el **coseno del ángulo** entre ellos. Para un cono de 90° en total (45° a cada lado):

```
const float HalfAngleRad = 45.0f * 3.14159265f / 180.0f; // a radianes (tema 30)
if (Dot(MyForward, ToTarget) > std::cos(HalfAngleRad)) // cos(45°) ≈ 0.707
{
    // dentro del cono: el enemigo me ve
}
```

El ángulo exacto, si lo necesitas: `float Angle = std::acos(D);` (en radianes).

Trampa: si no normalizas los dos vectores, el resultado se multiplica por sus longitudes y deja de ir de -1 a 1. Para saber solo "delante o detrás" basta el signo, pero para el cono **tienes que normalizar**.

28. Cross product (producto vectorial) [Explicarlo](#)

Toma dos vectores y devuelve **otro vector perpendicular a los dos**. Se usa para dos cosas: saber si algo está a la **izquierda o a la derecha**, y calcular la **normal** de una superficie (la dirección hacia la que "mira").

```
Vec3 Cross(const Vec3& a, const Vec3& b)
{
    return { a.Y * b.Z - a.Z * b.Y,
            a.Z * b.X - a.X * b.Z,
            a.X * b.Y - a.Y * b.X };
}
```

¿Izquierda o derecha? Lo más sencillo es verlo en 2D, mirando el mundo desde arriba (X a la derecha, Y hacia arriba del papel). Solo hace falta un número:

```
float Cross2D(const Vec2& a, const Vec2& b)
{
    return a.X * b.Y - a.Y * b.X;
}

float C = Cross2D(MyForward, ToTarget);
if (C > 0.0f) { /* el objetivo está a mi IZQUIERDA */ }
if (C < 0.0f) { /* está a mi DERECHA */ }
```

- Uso en combate: dot para "¿delante o detrás?" y cross para "¿izquierda o derecha?". Con los dos eliges la animación de reacción al golpe según de dónde viene.
- `Cross(A, B)` y `Cross(B, A)` dan vectores opuestos: el orden importa.

Trampa: en 3D, qué signo significa izquierda depende de qué eje es arriba y de la orientación de los ejes del motor. Compruébalo una vez con un caso conocido.

29. Interpolación (Lerp) [Escribirlo](#)

Lerp (interpolación lineal) da un punto **entre A y B**. $t = 0$ es A, $t = 1$ es B, $t = 0.5$ es justo la mitad.

```
float Lerp(float a, float b, float t)
{
    return a + (b - a) * t;
}

Lerp(0.0f, 100.0f, 0.25f);    // 25

Vec3 Lerp(const Vec3& a, const Vec3& b, float t)
{
    return a + (b - a) * t;    // la misma fórmula con vectores
}
```

Acercarse suavemente a un objetivo (cámara que sigue, barra de vida que baja poco a poco):

```
float T = std::min(Speed * DeltaTime, 1.0f);    // nunca más de 1
DisplayedHealth = Lerp(DisplayedHealth, RealHealth, T);    // cada frame, un trozo del camino
```

Trampa: con `t > 1` te pasas de B. Limita `t` a 1 (como arriba).

30. Ángulos y trigonometría Escribirlo

Los ángulos se miden en **grados** (0-360) o en **radianes** (0- 2π). Las funciones de C++ (`std::sin`, `std::cos`, `std::atan2`) trabajan **siempre en radianes**.

```
constexpr float Pi = 3.14159265f;

float ToRadians(float deg) { return deg * Pi / 180.0f; }
float ToDegrees(float rad) { return rad * 180.0f / Pi; }
```

Quiero...	Uso
la dirección de un ángulo	<code>Vec2 Dir{ std::cos(A), std::sin(A) };</code>
el ángulo de una dirección	<code>float A = std::atan2(Dir.Y, Dir.X);</code>
mover algo en círculo	<code>Pos = Center + Vec2{ std::cos(A), std::sin(A) } * Radius;</code> y sumar a <code>A</code> con <code>DeltaTime</code>

Girar hacia el objetivo (mirando desde arriba):

```
Vec2 ToTarget = Target - Me;
float TargetYaw = ToDegrees(std::atan2(ToTarget.Y, ToTarget.X));    // hacia dónde debería mirar
```

Trampa: `std::atan2` recibe **(y, x)**, en ese orden, no (x, y).

Trampa: los ángulos dan la vuelta: de 350° a 10° hay **20°**, no 340°. Para girar por el camino corto, deja la diferencia entre -180 y 180:

```
float DeltaAngle(float from, float to)    // ángulos entre 0 y 360
{
    return std::fmod(to - from + 540.0f, 360.0f) - 180.0f;
}

DeltaAngle(350.0f, 10.0f);    // 20
DeltaAngle(10.0f, 350.0f);    // -20
```

Rotaciones en 3D (nivel: reconocerlo)

- **Ángulos de Euler** (pitch, yaw, roll): tres ángulos, fáciles de entender. Sirven para leer y editar rotaciones.
- **Gimbal lock**: con Euler, en ciertas orientaciones dos ejes se alinean y pierdes un grado de libertad. Además, interpolar ángulos de Euler da giros raros.
- **Cuaterniones**: otra forma de guardar una rotación (4 números). No tienen gimbal lock e interpolan bien (**Slerp**). Los motores los usan por dentro. No hace falta saber su matemática, solo **cuándo** usarlos: para combinar e interpolar rotaciones.

31. Espacio local y espacio mundo [Explicarlo](#)

Una posición **en mundo** se mide desde el origen del mundo. Una posición **en local** se mide desde un objeto, con sus propios ejes: "1 metro delante de mí" es local. Para pasar de local a mundo hay que aplicar la **transformación** del objeto: su escala, su rotación y su posición.

Local (desde el personaje):	la punta de la espada está en (1, 0): 1 delante
Mundo:	depende de dónde está el personaje y hacia dónde mira

En 2D, sin escala: primero giras el punto local según la rotación del objeto y luego le sumas su posición.

```
Vec2 LocalToWorld(const Vec2& local, const Vec2& objectPos, float objectAngleRad)
{
    const float C = std::cos(objectAngleRad);
    const float S = std::sin(objectAngleRad);

    Vec2 Rotated{ local.X * C - local.Y * S,    // 1. girar
                  local.X * S + local.Y * C };

    return objectPos + Rotated;                // 2. mover
}
```

- El **forward** (hacia dónde mira un objeto) es su eje "delante" pasado a mundo: en 2D, { `std::cos(Angle)`, `std::sin(Angle)` }.
- Nivel reconocerlo: los motores guardan cada transformación en una **matriz 4x4**. Multiplicar matrices encadena transformaciones (el hijo respecto al padre, el padre respecto al mundo), y **el orden de la multiplicación importa**.

32. Colisiones básicas [Escribirlo](#)

Dos esferas se tocan si la distancia entre centros es menor o igual que la suma de radios. Se compara al cuadrado para no hacer la raíz:

```
bool SpheresOverlap(const Vec3& a, float ra, const Vec3& b, float rb)
{
    const float R = ra + rb;
    return (b - a).LengthSquared() <= R * R;
}
```

AABB (*Axis-Aligned Bounding Box*): una caja alineada con los ejes, definida por su esquina mínima y su esquina máxima. Dos cajas se tocan si se solapan **en los tres ejes a la vez**:

```
struct AABB
{
    Vec3 Min;
    Vec3 Max;
};

bool Overlaps(const AABB& a, const AABB& b)
{
    return a.Min.X <= b.Max.X && a.Max.X >= b.Min.X
        && a.Min.Y <= b.Max.Y && a.Max.Y >= b.Min.Y
        && a.Min.Z <= b.Max.Z && a.Max.Z >= b.Min.Z;
}
```

- Nivel explicarlo: un **raycast** lanza un rayo desde un punto en una dirección y devuelve lo primero que toca. Se usa para disparos, línea de visión y detección del suelo. El motor lo hace por ti.
- Por eso se usan cajas y esferas: comprobarlas es baratísimo. Primero una prueba simple para descartar, y solo si pasa, una prueba precisa.

Parte 4 — Preguntas de entrevista

Cómo usarla: tapa la respuesta, contesta **en voz alta** con tus palabras y compara. Lo que importa no es la frase exacta, sino que se note que entiendes el porqué.

Memoria

¿Diferencia entre stack y heap?

Respuesta: El stack guarda las variables locales: se crean y se destruyen solas al salir del scope, y es muy rápido. El heap es memoria que pides tú (`new`): vive hasta que alguien la libera, y es más lento. Uso el stack siempre que puedo y el heap cuando el objeto tiene que vivir más que su función, o cuando hay polimorfismo o cantidades decididas en runtime.

Ejemplo:

```
void SpawnBoss(Arena& Arena)
{
    Enemy Minion("Orc");                // stack: muere al salir de la función
    auto Boss = std::make_unique<Enemy>("Boss"); // heap: vive lo que decida su dueño
    Arena.Add(std::move(Boss));           // sobrevive a la función dentro de Arena
}
```

Por qué lo preguntan: es la base de todo lo demás: punteros, leaks, smart pointers. Si no lo tienes claro, lo demás no se sostiene.

¿Qué es un memory leak? ¿Cómo lo evitas?

Respuesta: Memoria del heap que sigue reservada, pero a la que ya no llega nadie, porque murió el último puntero que sabía dónde estaba. Se evita con RAII: no escribo `new/delete` a mano, sino smart pointers o contenedores, que liberan solos en su destructor.

Ejemplo:

```
// Problema: si Fail() es true, nunca se hace el delete
void Load()
{
    SaveData* Save = new SaveData();
    if (Fail()) return;           // LEAK
    delete Save;
}

// Solución: RAII. Se borra solo en cualquier salida
void Load()
{
    auto Save = std::make_unique<SaveData>();
    if (Fail()) return;           // OK
}
```

Por qué lo preguntan: la oferta de Larian lo dice literalmente: código "sin memory leaks". Quieren ver que lo evitas por diseño, no por disciplina.

¿Qué es un puntero colgante? Pon un ejemplo.

Respuesta: Un puntero que apunta a algo que ya se destruyó. Usarlo es comportamiento indefinido. Ejemplos: guardar un puntero a un elemento de un `vector` y luego hacer `push_back`, que puede mudar los elementos; o devolver un puntero o referencia a una variable local.

Ejemplo:

```
// Problema
std::vector<Item> Items = { Item(1) };
Item* First = &Items[0];
Items.push_back(Item(2));           // puede mudar el array a otra zona
First->GetId();                     // UB: First apunta al array viejo

// Solución: guardar el índice, no la dirección
size_t FirstIndex = 0;
Items.push_back(Item(2));
Items[FirstIndex].GetId();          // OK
```

Por qué lo preguntan: es el bug de memoria más común en juegos (objetos que mueren mientras alguien los apunta). Un buen ejemplo demuestra experiencia real.

¿Qué es RAI?

Respuesta: Atar la vida de un recurso a la de un objeto: el constructor lo pide y el destructor lo libera. Como el destructor se ejecuta solo al morir el objeto, el recurso se libera siempre, aunque haya `return` tempranos o excepciones. `vector`, `string` y `unique_ptr` funcionan así.

Ejemplo:

```
#include <fstream>

void LoadSave()
{
    std::ifstream File("save.txt"); // el constructor abre el archivo
    if (!File)
    {
        return;                     // el destructor lo cierra
    }
    // ... leer ...
}
```

Por qué lo preguntan: es LA idea de C++ para gestionar recursos. Saber explicarla demuestra que piensas en C++, no en Java con punteros.

¿unique_ptr, shared_ptr o weak_ptr? ¿Cuándo cada uno?

Respuesta: `unique_ptr` por defecto: un solo dueño, coste cero. `shared_ptr` solo si la propiedad es realmente compartida y no sabes quién morirá el último. `weak_ptr` para observar un objeto de un `shared_ptr` sin mantenerlo vivo: por ejemplo, el objetivo de un enemigo. Para usarlo, `lock()`.

Ejemplo:

```
std::vector<std::unique_ptr<Enemy>> Enemies; // el nivel es el ÚNICO dueño de sus enemigos
std::shared_ptr<Texture> Texture;           // varias mallas comparten la misma textura
std::weak_ptr<Player> Target;               // el enemigo OBSERVA al jugador, no es su dueño
```

Por qué lo preguntan: quieren ver que eliges por **propiedad** ("¿quién es el dueño?"), no que pones `shared_ptr` en todo por comodidad.

¿Para qué sirve `weak_ptr`?

Respuesta: Para dos cosas. Observar sin mantener vivo: si el enemigo guardara un `shared_ptr` al jugador, el jugador no moriría nunca mientras el enemigo exista. Y romper ciclos: si A y B se apuntan con `shared_ptr`, el contador nunca llega a 0 y es un leak; uno de los dos tiene que ser `weak_ptr`.

Ejemplo:

```
// Problema: ciclo. Squad -> Soldier y Soldier -> Squad con shared_ptr: nunca llegan a 0
struct Soldier { std::shared_ptr<Squad> MySquad; };

// Solución: el padre es dueño de los hijos; el hijo solo observa al padre
struct Squad { std::vector<std::shared_ptr<Soldier>> Members; };
struct Soldier { std::weak_ptr<Squad> MySquad; };
```

Por qué lo preguntan: separa a quien ha usado smart pointers de quien solo se sabe la definición.

Lenguaje

¿Puntero o referencia? ¿Cuándo cada uno?

Respuesta: Una referencia es un alias: no puede ser null ni reapuntarse. La uso cuando el objeto seguro que existe, por ejemplo en parámetros `const T&`. El puntero, cuando puede no haber objeto (`nullptr`) o necesito cambiar a qué apunta.

Ejemplo:

```
void ApplyDamage(Fighter& Target, float Amount); // siempre hay objetivo: referencia
Fighter* FindTarget();                          // puede no haber objetivo: puntero
(nullptr)
```

Por qué lo preguntan: es de las primeras preguntas de cualquier test de C++. Esperan la diferencia y un criterio de uso.

¿Para qué sirve `const` en un método? ¿Y en un parámetro `const&`?

Respuesta: En un método, promete que no modifica el objeto, y el compilador lo vigila. Además, solo se pueden llamar métodos `const` sobre objetos `const`. En un parámetro, `const T&` pasa el original sin copiarlo, y promete no modificarlo. Es lo normal para clases y strings.

Ejemplo:

```

class Fighter
{
public:
    float GetHealth() const;        // no modifica
    void TakeDamage(float amount); // modifica
};

void PrintHealth(const Fighter& F) // original, sin copia, solo lectura
{
    F.GetHealth();                // OK: es const
    F.TakeDamage(5.0f);           // ERROR de compilación: no es const
}

```

Por qué lo preguntan: la "const correctness" se nota enseguida en un code review. Demuestra cuidado con las interfaces.

¿Para qué sirve explicit?

Respuesta: Un constructor de un solo parámetro sirve también de conversión automática: sin `explicit`, `Equip(50.0f)` compila creando un `Weapon` temporal sin avisar. `explicit` prohíbe esa conversión. Lo pongo en todo constructor de un parámetro, salvo que quiera la conversión a propósito.

Ejemplo:

```

explicit Weapon(float damage);

void Equip(const Weapon& w);

Equip(50.0f);           // ERROR: sin explicit compilaría creando un Weapon oculto
Equip(Weapon(50.0f));   // OK: la conversión está a la vista

```

Por qué lo preguntan: es un detalle fino. Si lo sabes explicar, demuestras que conoces las trampas del lenguaje, no solo la sintaxis.

¿Diferencia entre struct y class?

Respuesta: Técnicamente solo el acceso por defecto: `public` en `struct` y `private` en `class`. Por convención, `struct` para datos simples y `class` para objetos con comportamiento y datos protegidos.

Ejemplo:

```

struct HitInfo                // datos simples: todo public por defecto
{
    float Damage = 0.0f;
    bool bIsCritical = false;
};

class Fighter                 // comportamiento + datos protegidos: private por defecto
{
public:
    void TakeDamage(float amount);
private:
    float Health = 100.0f;
};

```

Por qué lo preguntan: pregunta rápida para ver si sabes que no hay más diferencia que esa.

¿static_cast o dynamic_cast?

Respuesta: `static_cast` para conversiones normales que se conocen al compilar (por ejemplo `float` a `int`). `dynamic_cast` para bajar en una jerarquía comprobando en runtime: devuelve `nullptr` si el objeto no es de ese tipo. Requiere que la clase tenga algo `virtual` y cuesta algo más.

Ejemplo:

```
int Rounded = static_cast<int>(3.7f);           // 3: conversión conocida al compilar

if (Sword* S = dynamic_cast<Sword*>(W))        // ¿W es realmente una Sword?
{
    S->Parry();                                // sí: uso lo específico de Sword
}
```

Por qué lo preguntan: demuestra que sabes que hay casts seguros y que evitas el cast estilo C.

Herencia

¿Qué hace virtual? ¿Cómo funciona por dentro?

Respuesta: Hace que se llame a la versión del objeto real, aunque lo manejes con un puntero al padre, y se decide en runtime. Por dentro, cada clase con funciones virtuales tiene una tabla de punteros a funciones (la **vtable**), y cada objeto guarda un puntero oculto a la tabla de su clase. La llamada pasa por esa tabla. Tiene un coste pequeño, pero no es gratis.

Ejemplo:

```
std::unique_ptr<Weapon> W = std::make_unique<Sword>("Katana", 25.0f);
W->Use();    // el objeto Sword guarda un puntero a la vtable de Sword -> llama a Sword::Use
```

Por qué lo preguntan: el polimorfismo es la base de muchos sistemas de gameplay. Mencionar la vtable demuestra que entiendes lo que hay debajo.

¿Por qué el destructor de una clase base debe ser virtual?

Respuesta: Porque si borras un hijo a través de un puntero al padre (lo que hace un `unique_ptr<Weapon>` que guarda un `Sword`), sin `virtual` solo se ejecuta el destructor del padre. La parte del hijo no se destruye: es comportamiento indefinido y normalmente un leak. Con `virtual`, se ejecuta hijo → padre.

Ejemplo:

```
// Problema
class Weapon { public: ~Weapon(); };           // SIN virtual
std::unique_ptr<Weapon> W = std::make_unique<Sword>(...);
// al morir W: solo se ejecuta ~Weapon(). ~Sword() nunca -> UB

// Solución
class Weapon { public: virtual ~Weapon(); };   // ~Sword() y luego ~Weapon()
```

Por qué lo preguntan: es LA pregunta clásica de herencia. Casi fija en cualquier test.

¿Qué hace override?

Respuesta: Le dice al compilador: "esta función reemplaza una virtual del padre". Si te equivocas en la firma, o el padre no es `virtual`, da error. Sin `override`, crearías una función nueva sin enterarte.

Ejemplo:

```
class Weapon
{
public:
    virtual void Use() const;
};

class Sword : public Weapon
{
public:
    // (tres alternativas, solo una a la vez)
    void Use();                // sin const: es una función NUEVA. Compila y no avisa
    void Use() override;       // ERROR: el compilador ve que no coincide con la del padre
    void Use() const override; // correcto
};
```

Por qué lo preguntan: demuestra que conviertes errores silenciosos en errores de compilación.

¿Qué es una clase abstracta?

Respuesta: Una clase con al menos una función virtual pura (`= 0`). No se puede crear directamente: sirve de interfaz, y los hijos están obligados a implementar esas funciones.

Ejemplo:

```
class Weapon
{
public:
    virtual ~Weapon() = default;
    virtual void Use() const = 0;    // virtual pura
};

Weapon W("x", 1.0f);                // ERROR: Weapon es abstracta
std::unique_ptr<Weapon> P = std::make_unique<Sword>("Katana", 25.0f);    // OK
```

Por qué lo preguntan: base de las interfaces en C++, y del diseño de sistemas con varios tipos intercambiables.

¿Herencia o composición?

Respuesta: Herencia solo si hay relación "es un" y necesito polimorfismo. Si no, composición: la clase **tiene** otro objeto como miembro. La composición es más flexible y evita jerarquías profundas difíciles de cambiar.

Ejemplo:

```
// Mal: ¿un jugador "es un" inventario? No
class Player : public Inventory { };

// Bien: el jugador TIENE un inventario y TIENE vida
class Player
{
    Inventory Bag;
    HealthComponent Health;
};
```

Por qué lo preguntan: es una pregunta de criterio de arquitectura. Las jerarquías profundas son un problema clásico en código de juegos.

Copia y move

¿Qué pasa si copias una clase que tiene un puntero crudo dueño?

Respuesta: La copia por defecto copia la dirección, no lo apuntado. Los dos objetos apuntan al mismo array y creen que es suyo. Al morir, los dos hacen `delete`: double delete, comportamiento indefinido. Solución: usar `vector` o `unique_ptr` (regla de 0), o escribir la copia a mano (regla de 3/5), o prohibirla con `= delete`.

Ejemplo:

```
// Problema: la copia duplica la dirección -> double delete
class Inventory
{
    int* Slots;
};

// Solución (regla de 0): vector sabe copiarse; cada copia tiene su array
class Inventory
{
    std::vector<int> Slots;
};
```

Por qué lo preguntan: combina memoria, copia y RAII en una sola pregunta. Es muy típica.

¿Diferencia entre copiar y mover? ¿Qué hace `std::move`?

Respuesta: Copiar duplica los datos; mover se lleva los datos del otro y lo deja vacío, lo que es mucho más barato. `std::move` no mueve nada: solo marca el objeto como "se puede mover" (lo convierte a `&&`). El trabajo lo hacen el constructor y la asignación de move.

Ejemplo:

```
std::vector<int> A(1000000);

std::vector<int> B = A;           // copia: se duplican un millón de ints
std::vector<int> C = std::move(A); // move: C se queda el array de A. A queda vacío
```

Por qué lo preguntan: que `std::move` no mueve es la trampa favorita de los entrevistadores.

¿Qué es la regla de 0 / 3 / 5?

Respuesta: Si tu clase gestiona un recurso a mano y escribes una de las funciones especiales (destructor, copia o move), casi seguro necesitas todas: 3 en C++ antiguo, 5 con move. Lo ideal es la regla de 0: usar miembros que ya se gestionan solos y no escribir ninguna.

Ejemplo:

```
class UniqueSave
{
public:
    ~UniqueSave();                // gestiono un recurso a mano...
    UniqueSave(const UniqueSave&) = delete;    // ...así que decido qué pasa al
copiar
    UniqueSave& operator=(const UniqueSave&) = delete;
    UniqueSave(UniqueSave&&) noexcept;        // ...y al mover
    UniqueSave& operator=(UniqueSave&&) noexcept;
};
```

Por qué lo preguntan: demuestra que conoces la teoría **y** que en la práctica la evitas.

Lambdas y STL

¿Qué es una lambda? ¿Qué peligro tiene capturar por referencia?

Respuesta: Una función sin nombre que escribes donde la necesitas. Puede capturar variables por copia ([x]) o por referencia ([&x]). El peligro: si capturas por referencia (o `this`) y la lambda se ejecuta más tarde, como en un callback o un timer, esa variable u objeto puede haber muerto. Entonces es dangling.

Ejemplo:

```
// Problema: Damage muere al salir de la función, pero la lambda se ejecuta después
void SetupTimer(Timer& T)          // Timer = clase inventada con un std::function
{
    float Damage = 10.0f;
    T.OnFinished = [&Damage]() { ApplyDamage(Damage); };    // dangling
}

// Solución: capturar por copia
T.OnFinished = [Damage]() { ApplyDamage(Damage); };
```

Por qué lo preguntan: las lambdas están por todas partes en código de juego (callbacks, eventos). El peligro de la captura es un bug real y frecuente.

¿Por qué vector suele ser más rápido que una lista enlazada?

Respuesta: Porque guarda los elementos seguidos en memoria, y la CPU los lee muy rápido gracias a la caché. En una lista, cada nodo está en un sitio distinto del heap y cada salto es lento. Aunque la lista inserte en O(1) en teoría, en la práctica el `vector` suele ganar.

Ejemplo:

```
std::vector<Enemy> Enemies;    // [E][E][E][E]: seguidos, la CPU los carga en bloque
std::list<Enemy> Enemies;     // [E]->...->[E]->...->[E]: un salto por el heap por elemento
```

Por qué lo preguntan: en juegos el rendimiento importa. Demuestra que piensas en cómo funciona el hardware, no solo en la O grande.

¿Por qué un `push_back` puede romper un puntero a un elemento del vector?

Respuesta: Si el vector se queda sin espacio, reserva un array más grande, muda todos los elementos y libera el viejo. Los punteros, referencias e iteradores que apuntaban al array viejo quedan colgantes. Se evita con `reserve`, con índices o guardando `unique_ptr` en el vector.

Ejemplo:

```
std::vector<Item> Items;
Items.reserve(100);           // reservo antes: no habrá mudanza hasta el elemento 100
Items.push_back(Item(1));
Item* First = &Items[0];      // seguro mientras Size <= 100

// Alternativa: std::vector<std::unique_ptr<Item>>. Los Item no se mudan nunca
```

Por qué lo preguntan: es un bug de memoria real y sutil. Explicarlo demuestra que entiendes qué hace el vector por dentro.

¿Qué significa $O(1)$, $O(\log n)$ y $O(n)$?

Respuesta: Es la notación Big-O: cómo crece el coste de una operación cuando crece el número de elementos. $O(1)$ no depende del tamaño (acceder a `V[i]`). $O(n)$ crece igual que el tamaño (buscar uno por uno en un vector). $O(\log n)$ descarta la mitad en cada paso (buscar en un `map`): con un millón de elementos, unos 20 pasos.

Ejemplo:

```
std::vector<int> V(1000000);
V[500];                               // O(1): va directo a la posición

std::find(V.begin(), V.end(), 42);     // O(n): mira uno por uno

std::map<int, int> M;
M.find(42);                             // O(log n): el árbol descarta la mitad por
paso
```

Por qué lo preguntan: elegir contenedor y algoritmo es parte del día a día. Quieren ver que sabes qué cuesta lo que escribes, sobre todo en código que corre cada frame.

¿`unordered_map` o `map`?

Respuesta: `unordered_map` es una tabla hash: busca en $O(1)$ de media, pero sin orden. `map` es un árbol ordenado: busca en $O(\log n)$ y lo puedes recorrer en orden. Por defecto `unordered_map`, salvo que necesite el orden.

Ejemplo:

```
std::unordered_map<std::string, int> Ammo;    // "¿cuántas flechas tengo?": O(1), sin orden
std::map<int, std::string> Leaderboard;      // puntuaciones que quiero recorrer ordenadas
```

Por qué lo preguntan: quieren ver que eliges contenedor por el uso, no por costumbre.

Compilación y errores

¿Qué es un error de linker y por qué ocurre?

Respuesta: El compilador compila cada `.cpp` por separado y el linker los une. Un error de linker (`unresolved external symbol`) significa que algo está declarado, pero el linker no encuentra su definición. Causas típicas: falta el cuerpo de la función, el `.cpp` no está en el proyecto o la firma no coincide entre `.h` y `.cpp`.

Ejemplo:

```
// Fighter.h
class Fighter { public: void TakeDamage(float amount); };

// Fighter.cpp – Problema: falta "Fighter::". Esto es OTRA función, libre
void TakeDamage(float amount) { }

// -> LNK2019: unresolved external symbol Fighter::TakeDamage

// Solución
void Fighter::TakeDamage(float amount) { }
```

Por qué lo preguntan: todos los juniors se lo encuentran. Saber leerlo sin miedo ahorra horas.

¿Para qué sirve una forward declaration?

Respuesta: Para decir "esta clase existe" sin incluir su `.h` completo. Basta cuando solo usas punteros, referencias o smart pointers a ella. Reduce el tiempo de compilación, que en proyectos grandes es enorme, y evita includes circulares.

Ejemplo:

```
// Enemy.h
class Player;                                // basta: solo guardo un puntero
class Enemy
{
    std::weak_ptr<Player> Target;
};

// Enemy.cpp
#include "Player.h"                            // aquí sí: llamo a Target.lock()->GetName()
```

Por qué lo preguntan: en proyectos grandes los tiempos de compilación son un problema real, y la calidad de los includes se nota.

¿Qué es el comportamiento indefinido? Pon ejemplos.

Respuesta: Código cuyo resultado el estándar no define: puede funcionar, crashear o corromper memoria y fallar más tarde en otro sitio. Ejemplos: usar un puntero colgante o `nullptr`, leer fuera de un array, variables sin inicializar, double delete, destructor base no virtual.

Ejemplo:

```
// Problema
int Health;           // sin inicializar: puede valer cualquier cosa
if (Health > 0) { }    // UB

// Solución
int Health = 100;
```

Por qué lo preguntan: demuestra que entiendes por qué un bug de memoria puede aparecer lejos de su causa, y por qué hay que evitarlo por diseño.

Matemáticas

¿Cómo sabes si un enemigo está delante o detrás del jugador?

Respuesta: Con el dot product entre el forward del jugador y la dirección normalizada del jugador al enemigo. Si es mayor que 0, está delante; si es menor, detrás.

Ejemplo:

```
Vec3 ToEnemy = (Enemy - Player).Normalized();  
bool bInFront = Dot(PlayerForward, ToEnemy) > 0.0f;
```

Por qué lo preguntan: es la pregunta de matemáticas más típica de gameplay. Se usa para ataques por la espalda, bloqueos frontales y reacciones a golpes.

¿Cómo harías el cono de visión de un enemigo?

Respuesta: Dos comprobaciones. Distancia: la distancia al cuadrado menor que el alcance al cuadrado. Ángulo: el dot entre su forward y la dirección normalizada al objetivo, mayor que el coseno de la mitad del ángulo del cono. Primero la distancia, que es más barata. Después, un raycast para comprobar que no hay paredes en medio.

Ejemplo:

```
bool CanSee(const Vec3& Me, const Vec3& Forward, const Vec3& Target, float HalfAngleRad,  
float Range)  
{  
    Vec3 ToTarget = Target - Me;  
    if (ToTarget.LengthSquared() > Range * Range)                // 1. ¿está lo  
    bastante cerca?  
    {  
        return false;  
    }  
    return Dot(Forward, ToTarget.Normalized()) > std::cos(HalfAngleRad); // 2. ¿dentro del  
    ángulo?  
}
```

Por qué lo preguntan: combina vectores, dot, trigonometría y rendimiento en un problema real de IA.

¿Cómo sabes si un golpe viene de la izquierda o de la derecha?

Respuesta: Con el cross product entre mi forward y la dirección hacia el atacante. El signo dice el lado. En 2D, visto desde arriba, es un solo número; en 3D, miro la componente del eje vertical. Junto con el dot (delante o detrás), elijo la animación de reacción.

Ejemplo:

```
float C = Cross2D(MyForward, ToAttacker);  
if (C > 0.0f) { PlayReaction("Left"); }  
else          { PlayReaction("Right"); }
```

Por qué lo preguntan: comprueba que sabes para qué sirve el cross en la práctica, no solo su fórmula.

¿Por qué se multiplica por DeltaTime?

Respuesta: Porque `DeltaTime` es el tiempo desde el frame anterior. Si muevo una cantidad fija por frame, a más FPS el juego va más rápido. Multiplicando por `DeltaTime`, las velocidades son por segundo y el juego se comporta igual con cualquier framerate.

Ejemplo:

```
Pos.X += 5.0f; // depende del framerate: a 120 FPS va el doble de rápido
Pos.X += 300.0f * DeltaTime; // 300 unidades por segundo en cualquier framerate
```

Por qué lo preguntan: es un error clásico de principiante. Quieren ver que lo tienes interiorizado.

¿Para qué sirve normalizar un vector?

Respuesta: Para quedarte solo con la dirección (longitud 1) y descartar el tamaño. Si mueves algo con el vector sin normalizar, iría más rápido cuanto más lejos estuviera el objetivo. También hace falta antes de un dot, para que el resultado sea el coseno del ángulo. Ojo con el vector cero: no se puede normalizar.

Ejemplo:

```
Vec3 ToPlayer = Player - Enemy; // largo = la distancia (por ejemplo, 500)
Enemy = Enemy + ToPlayer * Speed * DeltaTime; // MAL: cuanto más lejos, más rápido
Enemy = Enemy + ToPlayer.Normalized() * Speed * DeltaTime; // BIEN: siempre a Speed
```

Por qué lo preguntan: casi todos los bugs de movimiento y de ángulos de un principiante vienen de normalizar mal o no normalizar.

¿Cómo compruebas si dos esferas chocan? ¿Por qué se compara al cuadrado?

Respuesta: Chocan si la distancia entre centros es menor o igual que la suma de radios. Se compara el cuadrado de la distancia con el cuadrado de la suma, porque da el mismo resultado y te ahorras la raíz cuadrada, que es más cara. Importa cuando se hace miles de veces por frame.

Ejemplo:

```
bool SpheresOverlap(const Vec3& a, float ra, const Vec3& b, float rb)
{
    const float R = ra + rb;
    return (b - a).LengthSquared() <= R * R; // sin std::sqrt
}
```

Por qué lo preguntan: una matemática con rendimiento: es el tipo de detalle que se espera en código de juego.

¿Qué es el gimbal lock? ¿Para qué sirven los cuaterniones?

Respuesta: Con ángulos de Euler, en ciertas orientaciones dos ejes de giro se alinean y pierdes un grado de libertad: eso es el gimbal lock. Además, interpolar Euler da giros raros. Los cuaterniones guardan la rotación de otra forma, no tienen ese problema e interpolan bien con Slerp. Uso Euler para leer y editar, y cuaterniones para combinar e interpolar.

Ejemplo:

```
// Nivel explicarlo: no se escribe a mano. Lo importante es saber cuándo usar cada cosa:  
// Euler (pitch, yaw, roll) -> leer y editar rotaciones  
// Cuaternión + Slerp -> combinar e interpolar rotaciones sin gimbal lock
```

Por qué lo preguntan: a un junior no le piden la matemática de los cuaterniones, pero sí saber por qué existen.